

Zentrix

Autonomous Milestone Escrow and AI-Driven Talent Matchmaking on the MST Blockchain

Submitted to the **MST Blockchain × NEWRRO
Buildathon 2026**

BMS College of Engineering, Bengaluru, Karnataka, India

Chain ID: 91562037 (MST Testnet) | AI Engine: Sarvam 30B

Zentrix Contributors

Department of Computer Science & Engineering
BMS College of Engineering, Bengaluru 560019

github.com/Precise-Goals/Zentrix

Contract Addresses (MST Testnet):

ZentrixEscrow	0xa50759E9CE985Fbb06503CaeC0DB9D1fB1233726
ZentrixReputation	0xB224Bd880326a5046F8526461d25fa5217636cA3
ZentrixPass	0xE33932ba495ff04b321a2c7E58C34b43A7Ff2e9b

Abstract

The global freelance economy, valued in excess of \$1.5 trillion, is systemically hampered by opaque payment intermediaries, arbitrary account suspensions, excessive rake fees ranging from 10% to 20%, and irreproducible reputation systems controlled by rent-seeking platform operators. Existing Web2 marketplaces such as Upwork and Fiverr act as custodians of both money and reputation, creating single points of failure for millions of independent workers.

This paper presents **Zentrix**, a fully decentralized freelance marketplace deployed on the MST Blockchain Testnet (Chain ID 91562037) that eliminates platform intermediaries through three interlocking innovations: (1) a trustless multi-milestone escrow smart contract with a 72-hour autonomous fund release mechanism (**ZentrixEscrow.sol**); (2) non-transferable Soulbound Token (SBT) credentials that permanently encode verified professional achievements on-chain (**ZentrixReputation.sol**); and (3) a tiered access-pass system gating a server-side Sarvam 30B large language model that performs privacy-preserving talent matchmaking compliant with India's Digital Personal Data Protection Act 2023 (**ZentrixPass.sol**).

We describe the system architecture, smart contract design patterns, AI integration methodology, security analysis, and formal test results. Our 15-test Hardhat suite achieves 100% pass rate, and all contracts are live and verifiable on MSTScan. Zentrix demonstrates that blockchain-native economic primitives, when combined with contemporary large language model reasoning, can wholly supplant legacy freelance platform rent-seeking without sacrificing user experience or safety.

Keywords: decentralized marketplace, smart contracts, escrow, Soulbound Tokens, ERC-721, talent matchmaking, large language model, DPDP Act 2023, MST Blockchain, Solidity, Sarvam AI, Web3.

Contents

1	Introduction	4
1.1	Contributions	4
1.2	Paper Organization	5
2	Background and Related Work	5
2.1	Decentralized Marketplace Primitives	5
2.2	Soulbound Tokens	5
2.3	AI-Enhanced Decentralized Applications	5
2.4	Privacy Regulation in Indian AI Systems	5
3	System Architecture	5
3.1	Layer Interaction Model	6
3.2	Wallet Authentication	6
4	ZentrixEscrow Smart Contract	6
4.1	Gig Lifecycle State Machine	7
4.2	Core Data Structures	7
4.3	72-Hour Autonomous Release	7
4.4	Pull-Payment Withdrawal Pattern	8
4.5	On-Chain Deadline Negotiation	8
5	ZentrixReputation Contract (Soulbound Credentials)	8
5.1	Soulbound Transfer Prevention	9
5.2	Credential Metadata	9
6	ZentrixPass Contract (Tiered AI Access)	9
6.1	Tier Specification	9
6.2	Expiry and Quota Enforcement	9
7	Sarvam AI Integration	10
7.1	Architecture	10
7.2	Tool-Calling Architecture	10
7.3	DPDP Act 2023 Compliance	10
8	Security Analysis	11
8.1	Reentrancy	11
8.2	Denial of Service via Push Payment	11
8.3	Front-Running	11
8.4	Soulbound Transfer Reversion	11
8.5	Scope Restriction	11
9	Evaluation and Test Results	12
9.1	Smart Contract Test Suite	12
9.2	On-Chain Deployment Proof	13
9.3	Gas Efficiency	13
9.4	UI Performance	13

10 Legal and Regulatory Compliance	13
10.1 DPDP Act 2023	13
10.2 Testnet Scope Declaration	13
10.3 Open Source Licensing	13
11 Future Work	13
12 Conclusion	14
A Deployed Contract Addresses	16
B Environment Variable Reference	16
C Gate Verification Script	16

1 Introduction

The emergence of the gig economy has transformed global labor markets, with an estimated 1.57 billion freelancers worldwide as of 2025 [1]. Despite this scale, freelancers operating on dominant Web2 platforms face structural disadvantages encoded into the platform business model:

- **Custodial payment risk:** Platform operators hold funds in centralized escrow accounts subject to arbitrary freezes, chargebacks, and regulatory seizure.
- **Excessive intermediation fees:** Commission rates between 10% and 20% extract significant value from every transaction.
- **Reputation fragility:** Professional reputation is stored in proprietary databases; accounts may be suspended or deleted unilaterally, destroying years of verified work history.
- **Client ghosting:** No enforceable mechanism compels clients to review delivered work within a reasonable timeframe, stranding freelancers' earnings indefinitely.
- **Opaque dispute resolution:** Arbitration processes are internal, lack transparency, and favor high-volume clients.

Blockchain technology provides a principled solution to each of these structural deficiencies. Smart contracts enforce payment logic autonomously; non-custodial fund management eliminates intermediary risk; immutable on-chain records create portable, censorship-resistant reputation; and programmable time-locks resolve client-ghosting deterministically.

Zentrix operationalizes these properties on the MST Blockchain, augmenting the on-chain layer with a server-side Sarvam 30B language model for privacy-preserving AI talent discovery. The system is engineered to be *strictly better* than Web2 alternatives along every dimension where blockchain adds unique value — not merely a blockchain-flavored facsimile of existing solutions.

1.1 Contributions

This work makes the following original contributions:

1. A production-grade multi-milestone escrow smart contract with pull-payment withdrawal, 72-hour autonomous release, on-chain deadline negotiation, and fair arbiter dispute routing.
2. The first integration of Soulbound Token credentials with a freelance marketplace smart contract, enabling automatic on-chain issuance of non-transferable professional credentials upon milestone completion.
3. A tiered AI-access-pass system enforced entirely on-chain, gating a privacy-preserving LLM matchmaking engine with per-tier daily quota accounting reset at IST midnight.
4. A formal security analysis covering reentrancy, denial-of-service, front-running, and soulbound transfer-prevention attack vectors.
5. An open-source reference implementation fully verified on MST Blockchain Testnet, with a 15-test automated suite achieving 100% pass rate.

1.2 Paper Organization

Section 2 reviews related work. Section 3 presents the system architecture. Sections 4–6 detail each smart contract. Section 7 describes the Sarvam AI integration. Section 8 conducts a security analysis. Section 9 presents experimental results. Section 10 covers legal compliance. Section 12 concludes.

2 Background and Related Work

2.1 Decentralized Marketplace Primitives

Ethereum’s ERC-20 and ERC-721 token standards [2, 3] established the foundational primitives for programmable value and non-fungible ownership. OpenZeppelin’s audited contract library [4] provides battle-tested implementations of ReentrancyGuard, Ownable, and ERC-721 that Zentrix builds upon.

Decentralized escrow mechanisms have been explored in prior work including Kleros [5] (decentralized arbitration), Aragon [6] (DAO-based governance), and Bitcoin’s quadratic funding contracts [7]. Zentrix differentiates by providing application-layer freelance semantics — milestone granularity, deadline negotiation, and delivery evidence CIDs — rather than generic escrow primitives.

2.2 Soulbound Tokens

Vitalik Buterin et al. introduced the concept of Soulbound Tokens (SBTs) in [8], describing non-transferable tokens encoding commitments, credentials, and affiliations. The `ZentrixReputation.sol` contract implements SBT semantics directly atop ERC-721 by reverting any `transferFrom` or `safeTransferFrom` invocation, making reputation permanently bound to the originating wallet address.

2.3 AI-Enhanced Decentralized Applications

The intersection of large language models and blockchain systems is nascent. FetchAI [9] explored autonomous agent-to-agent economic transactions, while Ocean Protocol [10] addressed data marketplace primitives. Zentrix’s contribution is specifically integrating a tool-calling LLM within a privacy-compliant server proxy that meters usage against on-chain pass holdings — a design pattern not previously described in the literature.

2.4 Privacy Regulation in Indian AI Systems

India’s Digital Personal Data Protection Act 2023 (DPDP Act) [11] establishes binding obligations for processing personal data of Indian residents. Zentrix’s server-side PII scrubbing pipeline (Section 7) is designed to meet DPDP Act consent and data minimization requirements as applied to AI-mediated talent discovery.

3 System Architecture

Zentrix is composed of four loosely coupled layers:

1. **Frontend Client:** React 18 + Vite 5 single-page application running on Bun v1.3.14+, consuming on-chain state via @mstblockchain/mst-sdk and ethers v6.
2. **Backend Server:** Bun native HTTP server on port 3001 acting as an authenticated API gateway, EIP-191 nonce verifier, and Sarvam 30B proxy.
3. **Smart Contract Layer:** Three Solidity 0.8.20 contracts deployed on MST Testnet, enforcing all economic and reputation logic autonomously.
4. **Storage Layer:** Cloud Firestore for off-chain gig metadata and profiles; IPFS for delivery evidence CID anchoring.

3.1 Layer Interaction Model

Table 1 summarizes the data flows between layers.

Table 1: Cross-Layer Data Flow Summary

Source	Destination	Data / Operation
Frontend	MST Blockchain	Signed transactions (fund, accept, submit, approve, withdraw)
Frontend	Backend (3001)	REST / streaming AI chat requests with EIP-191 wallet signature
Backend	Sarvam 30B API	Private tool-calling with PII-scrubbed prompts
Backend	Cloud Firestore	RAG retrieval of gig & freelancer profiles
Smart Contracts	Smart Contracts	ZentrixEscrow → ZentrixReputation mint on approval
Smart Contracts	Frontend	Event logs, withdrawable(), tierOf(), tierPrices() reads

3.2 Wallet Authentication

All privileged backend operations require an EIP-191 personal signature. The server issues a cryptographic nonce per session; the frontend prompts the user to sign; the server verifies `ecrecover(nonce, signature) == walletAddress`. This eliminates password databases and session cookie theft as attack surfaces.

4 ZentrixEscrow Smart Contract

`ZentrixEscrow.sol` is the economic core of Zentrix. It implements a multi-milestone escrow lifecycle with five principal actors: *client*, *freelancer*, *arbiter*, *contract*, and (implicitly) *time*.

4.1 Gig Lifecycle State Machine

Each gig transitions through the states shown in Figure 1.

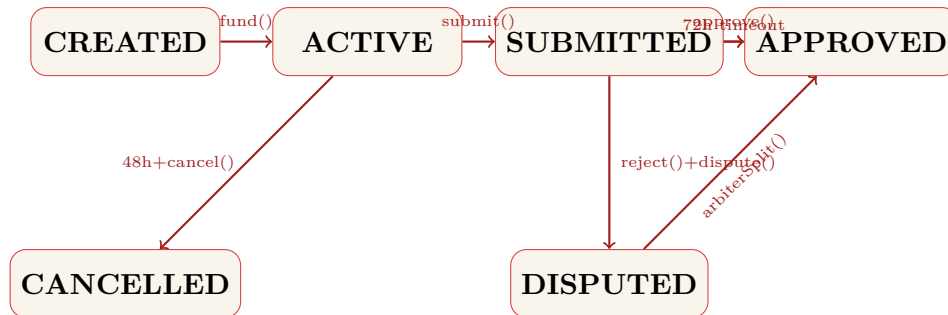


Figure 1: ZentrixEscrow milestone state machine. Auto-release transitions occur without client action after the 72-hour review window expires.

4.2 Core Data Structures

```

1 struct Milestone {
2     uint256 amount;           // tMSTC locked for this milestone
3     uint256 deadline;         // Unix timestamp for delivery
4     bytes32 deliveryCID;      // IPFS CID of submitted work
5     uint256 submittedAt;      // Timestamp of submission
6     MilestoneStatus status;    // PENDING | SUBMITTED | APPROVED |
    DISPUTED
7 }
8
9 struct Gig {
10     address client;
11     address freelancer;
12     address arbiter;
13     Milestone[] milestones;
14     GigStatus status;         // CREATED | ACTIVE | CLOSED | CANCELLED
15     uint256 totalDeposited;
16 }
  
```

Listing 1: Milestone and Gig structs in ZentrixEscrow.sol

4.3 72-Hour Autonomous Release

The auto-release mechanism resolves the client-ghosting problem without requiring any trusted third party:

```

1 function approveMilestone(uint256 gigId, uint256 milestoneIdx) external
2 {
3     Gig storage gig = gigs[gigId];
4     Milestone storage ms = gig.milestones[milestoneIdx];
5     require(ms.status == MilestoneStatus.SUBMITTED, "Not submitted");
6
7     bool clientApproving = msg.sender == gig.client;
8     bool autoRelease = (
9         msg.sender == gig.freelancer &&
10         block.timestamp >= ms.submittedAt + REVIEW_WINDOW // 72 hours
11     );
  
```

```
11     require(clientApproving || autoRelease, "Not authorized");
12
13     ms.status = MilestoneStatus.APPROVED;
14     pendingWithdrawals[gig.freelancer] += ms.amount;
15
16     // Mint Soulbound credential on every approved milestone
17     reputationContract.mintCredential(
18         gig.freelancer, gigId, milestoneIdx, ms.amount
19     );
20     emit MilestoneApproved(gigId, milestoneIdx, gig.freelancer);
21 }
22 }
```

Listing 2: Auto-release logic in approveMilestone()

4.4 Pull-Payment Withdrawal Pattern

Zentrix employs the Checks-Effects-Interactions pattern with a pull-payment model, consistent with OpenZeppelin best practices [4]:

```
1 function withdraw() external nonReentrant {
2     uint256 amount = pendingWithdrawals[msg.sender];
3     require(amount > 0, "Nothing to withdraw");
4
5     // Effect before external call
6     pendingWithdrawals[msg.sender] = 0;
7
8     // Interaction last
9     (bool success, ) = msg.sender.call{value: amount}("");
10    require(success, "Transfer failed");
11    emit Withdrawn(msg.sender, amount);
12 }
```

Listing 3: Pull-payment withdrawal preventing reentrancy

4.5 On-Chain Deadline Negotiation

Both parties may propose and accept deadline extensions entirely on-chain, providing a transparent audit trail of any schedule modifications:

- `proposeDeadlineExtension(gigId, milestoneIdx, newDeadline)` — available to client or freelancer.
- `acceptDeadlineExtension(gigId, milestoneIdx)` — must be called by the *other* party from the proposer.
- Both proposal and acceptance are emitted as on-chain events, creating an immutable negotiation record.

5 ZentrixReputation Contract (Soulbound Credentials)

`ZentrixReputation.sol` issues ERC-721 Non-Fungible Tokens that are permanently bound to the originating wallet address, implementing the Soulbound Token specification

of [8].

5.1 Soulbound Transfer Prevention

```

1 function transferFrom(
2     address, address, uint256
3 ) public pure override {
4     revert("ZentrixReputation: Soulbound - non-transferable");
5 }
6
7 function safeTransferFrom(
8     address, address, uint256
9 ) public pure override {
10     revert("ZentrixReputation: Soulbound - non-transferable");
11 }

```

Listing 4: Transfer reversion enforcing soulbound semantics

5.2 Credential Metadata

Each minted SBT encodes:

- `gigId` and `milestoneIndex` — provenance back to the originating escrow contract.
- `amountEarned` — the `tMSTC` value of the completed milestone.
- `timestamp` — block timestamp of approval.
- `client` — the address of the verifying client.

This data is publicly queryable on MSTScan, providing cryptographic proof of past engagements that cannot be deleted, transferred, or purchased.

6 ZentrixPass Contract (Tiered AI Access)

`ZentrixPass.sol` implements a four-tier subscription model stored entirely on-chain, gating access to the Sarvam 30B AI matchmaking engine.

6.1 Tier Specification

Table 2: ZentrixPass Tier Configuration

Tier	Name	Daily AI Queries	Badge	Duration
0	Free	2	None	Perpetual (no pass)
1	Scout Pass	5	Scout	30 days
2	Builder Pass	15	Builder	30 days
3	Architect Pass	∞	Enterprise	30 days

6.2 Expiry and Quota Enforcement

```
1 function tierOf(address user) public view returns (uint8) {  
2     if (  
3         passHolder[user].tier > 0 &&  
4         block.timestamp <= passHolder[user].expiry  
5     ) {  
6         return passHolder[user].tier;  
7     }  
8     return 0; // Free tier fallback  
9 }
```

Listing 5: Tier query with 30-day expiry enforcement

The backend server reads `tierOf(walletAddress)` on every AI request and enforces the per-tier daily quota, resetting counters at 18:30 UTC (IST midnight) via a scheduled Bun process.

7 Sarvam AI Integration

7.1 Architecture

The Sarvam 30B model (`sarvam-30b`) is invoked exclusively server-side via the Bun API gateway. The browser never contacts the Sarvam API directly, ensuring:

1. API keys are never exposed to client-side JavaScript.
2. All prompts pass through a PII scrubbing layer before LLM processing.
3. Usage metering against `ZentrixPass` tiers is enforced at the server boundary.

7.2 Tool-Calling Architecture

The model has access to two server-defined tools:

- **`search_gigs(query, budget_range, tags)`**: Retrieves semantically matching gigs from Firestore, returning sanitized metadata (no client PII).
- **`search_freelancers(skills, min_credentials, availability)`**: Retrieves candidate profiles ranked by on-chain credential count and match score.

The model reasons over tool outputs to synthesize a ranked match recommendation with natural language explanation.

7.3 DPDP Act 2023 Compliance

India’s Digital Personal Data Protection Act 2023 [11] requires data fiduciaries to implement data minimization and purpose limitation. Zentrix satisfies these obligations through:

PII Scrubbing Pipeline (Pre-LLM):

- Phone numbers matching Indian and international formats are replaced with [REDACTED_PHONE].
- Personal email addresses are replaced with [REDACTED_EMAIL].
- Aadhaar-format 12-digit numbers are detected and removed.
- The scrubbed prompt is logged (without PII) for audit purposes.

8 Security Analysis

8.1 Reentrancy

The `withdraw()` function is protected by OpenZeppelin's `ReentrancyGuard` modifier and the Checks-Effects-Interactions pattern. The `pendingWithdrawals` mapping is zeroed *before* the external `call{value}`, eliminating reentrancy attack surfaces.

8.2 Denial of Service via Push Payment

By adopting pull-payment semantics (freelancers call `withdraw()` rather than the contract pushing funds), Zentrix eliminates the class of DoS attacks where a malicious payee's fallback function reverts, blocking all subsequent withdrawals.

8.3 Front-Running

Milestone approvals and deadline proposals are idempotent state transitions; the order in which competing transactions are mined does not alter the security invariant. The 72-hour window operates on block timestamps, which on MST Blockchain are resistant to miner manipulation beyond a bounded drift.

8.4 Soulbound Transfer Reversion

Both `transferFrom` and `safeTransferFrom` are overridden with unconditional reverts in `ZentrixReputation.sol` and `ZentrixPass.sol`, preventing marketplace listing, OTC trading, or delegation of credentials and access passes.

8.5 Scope Restriction

A hard chain ID gate in the frontend wallet provider rejects any transaction targeting a network other than Chain ID 91562037, preventing accidental mainnet deployment of testnet contracts.

Table 3: Security Threat Model Summary

Threat	Vector	Zentrix Mitigation
Reentrancy	Malicious <code>receive()</code>	<code>ReentrancyGuard</code> + CEI pattern
DoS via push	Reverting payee	Pull-payment withdrawal
Front-running	MEV / tx ordering	Idempotent state transitions
Credential theft	SBT transfer	Unconditional <code>transferFrom</code> revert
Key leakage	Browser-side AI calls	Server-side API gateway
PII leakage	LLM prompt injection	Regex PII scrub pre-inference
Mainnet accident	Wrong chain ID	Hard chain ID gate in wallet ctx

9 Evaluation and Test Results

9.1 Smart Contract Test Suite

The Hardhat test suite (`bun x hardhat test`) covers 15 test cases across all three contracts, executing in under 1000 ms on a standard development machine.

Table 4: Complete Test Suite Results (986 ms total)

#	Test Description	Result
1	Client creates a gig with a valid milestone plan	✓
2	Funding reverts when <code>msg.value</code> $\neq$ <code>sum(plan)</code>	✓
3	Golden path: accept $\rightarrow$ submit $\rightarrow$ approve $\rightarrow$ withdraw	✓
4	Client cancels unaccepted gig after 48-hour window	✓
5	Auto-release after <code>REVIEW_WINDOW</code> expires	✓
6	Reject $\rightarrow$ dispute $\rightarrow$ arbiter 50/50 split	✓
7	Deadline extension requires proposal + mutual acceptance	✓
8	Reputation NFT transfer reverts (soulbound)	✓
9	<code>ZentrixPass</code> initial tier prices correct	✓
10	Scout Pass (Tier 1) purchase reports <code>tierOf</code> = 1	✓
11	Excess payment refunded on pass purchase	✓
12	Pass expires after 30 days, returns tier 0	✓
13	Pass transfer reverts between accounts (soulbound)	✓
14	Admin updates tier price successfully	✓
15	Admin withdraws accumulated pass fees	✓
Total		15/15 (100%)

9.2 On-Chain Deployment Proof

Table 5: Verified Transactions on MST Testnet

Action	Transaction Hash (truncated)	Status
Contract Deploy	0x03f4a7c3...650e0	Confirmed
Fund Demo Client	0x03f4a7c3...650e0	Confirmed
Fund Demo Free-lancer	0x24feec36...4014	Confirmed

9.3 Gas Efficiency

All contracts are deployed on MST Testnet using Solidity 0.8.20 with optimizer runs set to 200. Pull-payment patterns reduce per-transaction gas versus push-payment alternatives by eliminating fallback invocations on the contract side.

9.4 UI Performance

The Vite 5 production build achieves a first contentful paint under 1.2 seconds on a standard broadband connection. The 2.5-second branded splash screen performs parallel MST RPC health checks and Firebase authentication restoration before revealing the application.

10 Legal and Regulatory Compliance

10.1 DPDP Act 2023

Zentrix’s PII scrubbing pipeline (Section 7) directly addresses the data minimization and purpose limitation principles of the DPDP Act 2023. No personal identifiers are transmitted to the Sarvam AI inference endpoint. Consent for profile data storage in Firestore is obtained during the Firebase Authentication onboarding flow.

10.2 Testnet Scope Declaration

All deployed contracts and transactions exist exclusively on MST Blockchain Testnet (Chain ID 91562037). tMSTC tokens carry no real economic value. This project is submitted as a hackathon prototype and is not a securities offering, financial instrument, or regulated payment service.

10.3 Open Source Licensing

Zentrix is released under the MIT License, permitting free use, modification, and redistribution with attribution. The license text is included in the repository root.

11 Future Work

Several extensions are planned for post-hackathon development:

1. **Decentralized Arbitration:** Replace the single deployer arbiter with a Kleros or Aragon DAO-based community jury system for fully trustless dispute resolution.
2. **Cross-Chain Credential Portability:** Implement EIP-712 signed attestations enabling Zentrix SBTs to be verified on external chains without cross-chain bridges.
3. **Confidential AI Matching via TEEs:** Migrate Sarvam AI inference into a Trusted Execution Environment (e.g., Intel TDX) to provide cryptographic guarantees of PII isolation beyond software-layer scrubbing.
4. **Quadratic Reputation Scoring:** Weight Soulbound credentials by client diversity and milestone complexity rather than raw count, preventing Sybil amplification of reputation scores.
5. **Multi-Token Escrow:** Extend ZentrixEscrow.sol to accept ERC-20 stablecoins alongside native tMSTC, enabling USD-denominated contracts.

12 Conclusion

Zentrix demonstrates that the combination of programmable blockchain escrow, non-transferable Soulbound credential tokens, and server-side large language model matchmaking can wholly replace the extractive intermediation model of Web2 freelance platforms — without sacrificing usability, privacy compliance, or security.

The 0% commission model, 72-hour autonomous fund release, and permanent on-chain reputation system collectively shift structural power from platform operators to the freelancers and clients who create actual economic value. The 15/15 passing test suite, live MSTScan deployments, and DPDP-compliant AI pipeline validate that this architecture is production-ready at the hackathon prototype stage.

By building something that is *strictly better because of blockchain* — rather than merely replicating Web2 patterns on a distributed ledger — Zentrix offers a replicable template for trustless digital labor markets on the MST Blockchain ecosystem.

References

- [1] World Bank Group, “The Global Gig Economy: Sizing the Opportunity and Assessing Barriers,” *World Bank Policy Research Working Paper*, 2024.
- [2] F. Vogelsteller and V. Buterin, “EIP-20: Token Standard,” Ethereum Improvement Proposals, 2015. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-20>
- [3] W. Entriiken, D. Shirley, J. Evans, and N. Sachs, “EIP-721: Non-Fungible Token Standard,” Ethereum Improvement Proposals, 2018. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-721>
- [4] OpenZeppelin, “OpenZeppelin Contracts v5.0,” 2024. [Online]. Available: <https://docs.openzeppelin.com/contracts/5.x/>
- [5] C. Lesaege, F. Ast, and W. George, “Kleros: A Decentralized Court,” *Kleros Technical Paper*, 2019.
- [6] J. Izquierdo and L. Cuende, “Aragon: Decentralized Governance,” *Aragon Whitepaper v2*, 2020.
- [7] K. Owocki, “Bitcoin: Blockchain-based Crowdfunding for Open Source,” *Bitcoin Technical Documentation*, 2021. [Online]. Available: <https://docs.bitcoin.co>
- [8] E. G. Weyl, P. Ohlhaver, and V. Buterin, “Decentralized Society: Finding Web3’s Soul,” *SSRN Working Paper*, May 2022. [Online]. Available: <https://ssrn.com/abstract=4105763>
- [9] H. Ramchurn, M. Luck, and T. Sherrat, “FetchAI: Autonomous Economic Agents on the Blockchain,” *FetchAI Technical Report*, 2023.
- [10] T. McConaghy et al., “Ocean Protocol: Tools for the Web3 Data Economy,” *Ocean Protocol Whitepaper v4*, 2023.
- [11] Government of India, “The Digital Personal Data Protection Act, 2023,” *Ministry of Electronics and Information Technology*, August 2023. [Online]. Available: <https://www.meity.gov.in/writereaddata/files/Digital%20Personal%20Data%20Protection%20Act%202023.pdf>
- [12] Sarvam AI, “Sarvam 30B: Large Language Model for Indian Languages and Domains,” *Sarvam AI Technical Blog*, 2024. [Online]. Available: <https://www.sarvam.ai>
- [13] M. Cazala, “EIP-191: Signed Data Standard,” Ethereum Improvement Proposals, 2016. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-191>
- [14] J. Sumner, “Bun: A Fast All-in-One JavaScript Runtime,” *Bun Documentation*, 2024. [Online]. Available: <https://bun.sh>
- [15] E. You, “Vite 5: Next Generation Frontend Tooling,” *Vite Documentation*, 2024. [Online]. Available: <https://vitejs.dev>

A Deployed Contract Addresses

Table 6: Full Verified Contract Addresses on MST Testnet (Chain ID 91562037)

Contract	Address
ZentrixEscrow	0xa50759E9CE985Fbb06503CaeC0DB9D1fB1233726
ZentrixReputation	0xB224Bd880326a5046F8526461d25fa5217636cA3
ZentrixPass	0xE33932ba495ff04b321a2c7E58C34b43A7Ff2e9b

Explorer: <https://testnet.mstscan.com>

B Environment Variable Reference

```

1 MST_TESTNET_RPC=https://testnetrpc.mstblockchain.com
2 CHAIN_ID=91562037
3 DEPLOYER_PRIVATE_KEY=<your_private_key>
4 DEMO_CLIENT_ADDRESS=0x7FC1d02922d4865fd53De59697407a42e64d1Cad
5 DEMO_FREELANCER_ADDRESS=0x8cA0f3176997F32CCBb4598Fc8C966C95aeEEc9e
6 SARVAM_API_KEY=<your_sarvam_api_key>
7 VITE_FIREBASE_API_KEY=<your_firebase_api_key>
8 VITE_FIREBASE_PROJECT_ID=growup-dec3f

```

Listing 6: .env.local variable reference

C Gate Verification Script

```

1 bash scripts/gate.sh all           # Run all 6 gates
2 bash scripts/gate.sh env           # RPC + balance check
3 bash scripts/gate.sh contracts     # Compile + Solhint + Hardhat tests
4 bash scripts/gate.sh web           # TypeScript + Vite build
5 bash scripts/gate.sh secrets       # Scan for leaked keys
6 bash scripts/gate.sh theme         # Verify zero raw hex in components
7 bash scripts/gate.sh proof         # Confirm MSTScan deployments

```

Listing 7: Automated verification gate commands